Đề bài: Xây dựng Order Service trong hệ thống Microservices

Mục tiêu:

- Hiểu và áp dụng kiến thức về Spring Boot để xây dựng một microservice quản lý đơn hàng.
- Sử dụng **Spring Data JPA** để lưu trữ thông tin đơn hàng trong cơ sở dữ liệu quan hệ (MySQL hoặc PostgreSQL).
- Tích hợp với **Eureka Discovery Server** để đăng ký và quản lý service.
- Sử dụng **Resilience4j** để xử lý lỗi và tăng tính ổn định của hệ thống.
- Tích hợp **Kafka** để gửi thông báo khi đơn hàng được đặt thành công.
- Thực hành viết unit test và integration test.

Yêu cầu chi tiết:

1. Tạo ứng dụng Spring Boot

- Sử dụng **Spring Initializr** để khởi tạo dự án với các dependency:
  - Spring Web
  - Spring Data JPA
  - Spring Cloud Eureka Client
  - Spring Cloud Circuit Breaker (Resilience4j)
  - Spring Kafka
  - Lombok
  - Spring Boot Actuator
  - Micrometer Tracing (Zipkin)
  - Micrometer Registry Prometheus
- Cấu hình ứng dụng để kết nối với cơ sở dữ liệu (MySQL hoặc PostgreSQL).

2. Xây dựng các REST API

- **Đặt đơn hàng**:
  - API: `POST /api/order`
  - Request body:

```
{
 "orderLineItemsDtoList": [
  {
   "skuCode": "iphone_13",
   "price": 1200,
   "quantity": 1
  }
```

```
 ]
}
```

  - Response: HTTP Status 201 (Created) và thông báo "Order Placed".
- **Xử lý lỗi**:
  - Nếu sản phẩm không có trong kho (inventory), trả về thông báo lỗi và không tạo đơn hàng.
  - Sử dụng **Resilience4j** để xử lý lỗi khi gọi dịch vụ Inventory Service.

3. Sử dụng cơ sở dữ liệu quan hệ

- Tạo model `Order` và `OrderLineItems` với các trường:
  - `Order`:
    - `id` (Long)
    - `orderNumber` (String)
    - `orderLineItemsList` (List<OrderLineItems>)
  - `OrderLineItems`:
    - `id` (Long)
    - `skuCode` (String)
    - `price` (BigDecimal)
    - `quantity` (Integer)
- Sử dụng **Spring Data JPA** để thao tác với cơ sở dữ liệu.

4. Tích hợp với Eureka Discovery Server

- Cấu hình `application.properties` để đăng ký service với Eureka Server:

```
eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
spring.application.name=order-service
```

- Đảm bảo service có thể được discovery bởi các service khác trong hệ thống.

5. Tích hợp Kafka để gửi thông báo

- Khi đơn hàng được đặt thành công, gửi thông báo đến **Kafka** với topic `notificationTopic`.
- Sử dụng **Spring Kafka** để tích hợp với Kafka.

6. Sử dụng Resilience4j để xử lý lỗi

- Cấu hình **Resilience4j** để xử lý lỗi khi gọi dịch vụ Inventory Service:
  - Circuit Breaker: Đóng mạch khi tỷ lệ lỗi vượt quá ngưỡng.
  - Retry: Thử lại khi gọi dịch vụ thất bại.
  - Timeout: Giới hạn thời gian chờ phản hồi từ dịch vụ.

7. Tích hợp Actuator và Prometheus

- Cấu hình Actuator để expose các endpoint:
  - Health check: `/actuator/health`
  - Metrics: `/actuator/prometheus`
- Sử dụng **Micrometer** để tích hợp với Prometheus và Zipkin.

8. Viết unit test và integration test

- Viết test cho các API:
  - Test đặt đơn hàng thành công.
  - Test đặt đơn hàng thất bại do sản phẩm không có trong kho.
- Đảm bảo test coverage tối thiểu 80%.

9. Triển khai ứng dụng với Docker

- Tạo Dockerfile để đóng gói ứng dụng.
- Sử dụng Docker Compose để chạy ứng dụng cùng với cơ sở dữ liệu, Eureka Server, và Kafka.

Gợi ý:

1. **Công cụ và công nghệ sử dụng:**
   - Spring Boot
   - Spring Data JPA
   - Spring Cloud Eureka Client
   - Spring Cloud Circuit Breaker (Resilience4j)
   - Spring Kafka
   - Lombok
   - Actuator, Prometheus, Zipkin
   - Docker
2. **Tài liệu tham khảo:**
   - Tài liệu chính thức của Spring Boot: https://spring.io/projects/spring-boot
   - Hướng dẫn sử dụng Spring Data JPA: https://spring.io/guides/gs/accessing-data-jpa/
   - Hướng dẫn sử dụng Resilience4j: https://resilience4j.readme.io/
   - Hướng dẫn sử dụng Spring Kafka: https://spring.io/guides/gs/messaging-kafka/
3. **Cấu trúc dự án tham khảo:**

Copy

```
order-service/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com.hdbank.orderservice/
│   │   │   │   ├── controller/
│   │   │   │   ├── model/
│   │   │   │   ├── repository/
│   │   │   │   ├── service/
│   │   │   │   ├── dto/
│   │   │   │   ├── listener/
│   │   │   │   ├── config/
│   │   │   │   └── OrderServiceApplication.java
│   │   └── resources/
│   │       ├── application.properties
│   │       └── banner.txt
│   └── test/
│       └── java/
│           └── com.hdbank.orderservice/
│               └── OrderServiceApplicationTests.java
└── pom.xml
```

Phần mở rộng (nếu có thời gian):

- Thêm tính năng **Logging** và **Tracing** sử dụng **ELK Stack** hoặc **Jaeger**.
- Triển khai ứng dụng lên **Kubernetes**.
- Thêm tính năng **Authentication** và **Authorization** sử dụng **Spring Security**.